Remember, you can get to Django's version of the REPL using the `manage.py shell` command. This time, instead of creating a `Template` object, load one using the template engine based on your experiment.html file.

Django supports multiple template engines. You retrieve the one currently configured in your settings using `Engine.get_default()`. `Engine` objects have a `get_template()` method that loads a template similar to the `render()` shortcut.

Once you have a `Template` object, call `.render()`, passing in a `Context` object, like you did earlier in the chapter. Do this with different values for `instrument` to see the results of auto-escaping. The following listing renders the template twice, once with `trombone` and once with `tuba > baritone`.

Listing 3.11 Experimenting with special HTML characters in the Django REPL

```
>>> from django.template import Engine, Context
>>> engine = Engine.get_default()
>>> t = engine.get_template("experiment.html")
>>> data = {"instrument":"trombone"}
>>> t.render(Context(data))
'I love the sound a trombone makes.\n'
>>> data = {"instrument":"tuba > baritone"}
>>> t.render(Context(data))
'I love the sound a tuba &gt; baritone makes.\n'
```

Note how changing the value of `instrument` from `trombone` to `tuba > baritone` causes auto-escaping to execute. The greater-than symbol (`>`) is rendered as `>`, making it safe to display in HTML.

Django gives you several ways to control auto-escaping. The `| safe` filter indicates that a value should not be escaped. You do this if what you're rendering has HTML in it and you want it to display as HTML rather than be escaped. The `| escape` filter does the opposite, causing the filtered value to be auto-escaped. The `{% autoescape %}` tag allows you to turn auto-escaping on or off for a block. To see these in action, add a few more lines to experiment.html:

```
I love the sound a {{ instrument }} makes.
A shiny {{ instrument | safe }} is good.

{% autoescape off %}
Don't put a {{ instrument }} too close to your ear.
{% endautoescape %}
```

The template engine compiles and caches templates; if you modify experiment.html, you may or may not get the latest copy in the REPL. The Django development server watches your files and will reload edited templates, but that isn't guaranteed in the REPL. Exit the REPL after making the previous changes, and then run it again to see the effect of your new tags. Your results should look like those in the following listing. When you're done, don't close your REPL session; you're going to play around some more, using the same template in a moment.